Empirical Performance and Memory Evaluation of Canonical Matrix Multiplication Across C, Java, and Python

Pablo Martínez Suárez

Department of Computer Science and Systems, Universidad de Las Palmas de Gran Canaria (ULPGC)
Grado en Ingeniería y Ciencia de Datos · Big Data (2026–2027)

GitHub Repository: https://github.com/pablomrtinezzz/big-data-assignments

Abstract — Dense matrix multiplication is a foundational computational kernel in large-scale numerical linear algebra and data engineering. This paper presents a controlled empirical investigation comparing the canonical triple-loop algorithm ($i-j-k$) implemented in C, Java, and Python under identical experimental constraints. Across square matrix dimensions $n \in [32, 2048]$ with 64-bit floating-point arithmetic and deterministic pseudo-random inputs, we evaluate kernel execution time, throughput, variability, and memory footprint. We assess two C optimization levels (Clang `-O2` and `-O0`), the HotSpot JVM (OpenJDK 22 with C2 tiered compilation), and CPython 3.14.7. While all implementations scale with asymptotic cubic growth ($\alpha \approx 3.11 - 3.37$, $R^2 > 0.99$), execution constant factors diverge by up to 160 ×. On the evaluated Apple M2 platform, warm-up enables Java's JIT compiler to reach performance parity with optimized C, whereas CPython is severely bottlenecked by dynamic type dispatch and object boxing. Under a 60-second budget per repetition, Python is constrained to $n \leq 1024$, whereas C and Java execute up to $n = 2048$. Reproducibility artifacts, source code, and raw measurement logs are publicly available on GitHub at https://github.com/pablomrtinezzz/big-data-assignments.

Keywords — Matrix multiplication, algorithmic scaling, memory layout, C, Java, Python, benchmarking protocol, cache locality, JIT compilation.

1 Introduction & Research Objective

Matrix multiplication is a cornerstone operation in computational science, machine learning, and big data processing [1]. The standard mathematical definition of multiplying two dense square matrices $A, B \in \mathbb{R}^{n \times n}$ produces a matrix $C = AB$ where:

$$C_{ij} = \sum_{k=0}^{n-1} A_{ik} B_{kj}$$

This formulation implies an inherent arithmetic complexity of $O(n^3)$ operations—specifically n^3 multiplications and n^3 additions, yielding $2n^3$ floating-point operations (FLOPs).

While modern production libraries (e.g., OpenBLAS, Intel MKL, Apple Accelerate) achieve peak hardware throughput through architecture-specific vectorization (SIMD), cache-tiling, and multithreading [2], understanding the baseline performance of the naive triple-loop is essential for systems analysis. Programming language runtimes exhibit profound architectural divergence:

1. **Ahead-of-Time (AOT) Compiled Languages (C):** Direct translation into native machine instructions with manual memory layout and static optimization [3].
2. **Just-in-Time (JIT) Managed Environments (Java):** Bytecode execution transitioning dynamically via tiered profiling to aggressive native machine code compilation (HotSpot C2 compiler) [4].
3. **Interpreted Dynamic Languages (Python):** Bytecode evaluation loop (`ceval.c`) with dynamic type dispatch, boxing of primitives, and automatic memory management [5].

1.1 Research Question & Hypotheses

This investigation addresses the following core research question:

How does matrix dimension n govern execution time and memory footprint across AOT-compiled, JIT-managed, and interpreted execution environments when algorithmic structure, numeric precision, and input data are held strictly constant?

We formulate four testable empirical hypotheses:

- $\boldsymbol{H_1}$ **(Algorithmic Scaling):** All implementations will exhibit an empirical scaling exponent $\alpha \approx 3.0$ matching the theoretical $O(n^3)$ model, but interpreter overhead and dynamic type resolution will shift Python's constant factor c by over an order of magnitude relative to C.
- $\boldsymbol{H_2}$ **(JIT Compiler Convergence):** Following sufficient warm-up iterations, Java's HotSpot C2 compiler will achieve execution times within a factor of 1.5 × of optimized C (`-O2`), vastly outperforming unoptimized C (`-O0`).
- $\boldsymbol{H_3}$ **(Memory Layout Divergence):** Memory consumption will scale quadratically ($O(n^2)$). Due to `PyFloatObject` boxing and list pointer overhead, Python will exhibit a theoretical 4 × data storage expansion (32 bytes vs. 8 bytes per element) and substantially higher process resident memory, whereas C will allocate flat continuous buffers with minimal footprint overhead.
- $\boldsymbol{H_4}$ **(Feasible Operational Boundaries):** Under a declared execution budget of 60 seconds per repetition, Python will reach its feasibility limit at $n \leq 1024$, triggering timeouts at $n = 1536$, whereas C and Java will scale feasibly to $n \geq 2048$.

2 Background & Theoretical Foundations

2.1 The Canonical $i-j-k$ Loop Order

The canonical algorithm nests three loops in the row-column-dot order:

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        double sum = 0.0;
        for (int k = 0; k < n; k++) {
            sum += A[i * n + k] * B[k * n + j];
        }
        C[i * n + j] = sum;
    }
}
```

In this arrangement, the accumulator `sum` can reside in an architectural register during the innermost loop over k. However, access patterns to memory differ fundamentally across the three operands:

- **Matrix A:** Elements $A[i][k]$ are traversed sequentially along row i. Because data is stored in row-major order, consecutive iterations access consecutive memory addresses, maximizing **spatial cache locality**.
- **Matrix C:** Destination element $C[i][j]$ remains invariant during the inner loop and is updated once in the middle loop.
- **Matrix B:** Elements $B[k][j]$ are accessed along column j. In row-major layout, subsequent elements are separated by a stride of n doubles ($8n$ bytes). When $8n$ exceeds the cache line size (64 bytes), every iteration of k triggers a cache line request, leading to severe cache miss penalties once $n \times n$ exceeds the L1/L2 cache capacities [6].

2.2 Storage Layouts Across Paradigms

Memory organization dictates cache hierarchy interaction [7]:

1. **C (Contiguous 1D Flat Array):** Matrices are allocated as a single continuous memory block of n^2 doubles (`double*`) indexed as $i \cdot n + k$. No pointer indirection exists; element offsets are computed via affine arithmetic. Total data payload is exactly $24n^2$ bytes for A, B, C.
2. **Java (2D Array `double[][]`):** Java does not support true multidimensional contiguous primitive arrays. A `double[][]` is an array of n object references, each pointing to an independently allocated heap array of n primitive doubles. Each row array incurs an object header (16 bytes) plus length field (4 bytes) and 64-bit alignment padding [4]. Accessing $A[i][k]$ requires one pointer dereference to fetch the row array followed by index bounds checking.
3. **Python (Nested Lists `list[list[float]]`):** Standard Python matrices consist of an outer list containing references to n inner lists. Each inner list contains n pointers to heap-allocated `PyFloatObject` instances. In CPython 64-bit, each `PyFloatObject` requires 24 bytes (8B reference count, 8B type pointer, 8B double-precision value), supplemented by an 8-byte pointer in the enclosing list [5]. Consequently, each floating-point entry consumes 32 bytes—4 × the size of a primitive double—with double pointer dereferencing and complete destruction of cache spatial locality.

3 Experimental Methodology

3.1 Execution Environment & Hardware

All experiments were conducted on a dedicated Apple Silicon hardware platform running macOS Darwin:

- **Processor:** Apple M2 (ARMv8.5-A architecture)
  - 4 Performance cores (`Avalanche`, 3.50 GHz, 128 KB L1I, 64 KB L1D per core, 16 MB shared L2 cache)
  - 4 Efficiency cores (`Blizzard`, 2.42 GHz, 64 KB L1I, 32 KB L1D per core, 4 MB shared L2 cache)
  - System topology: 8 physical cores, 8 logical processors (1 thread/core, no SMT); executed single-threaded (thread count = 1).
- **Memory:** 8.0 GB Unified Memory (LPDDR5-6400, 100 GB/s bandwidth)
- **Operating System:** macOS Darwin 25.6.0 (Kernel 26.6.2, Build 25G83)
- **Software Toolchains:**
  - C Compiler: Apple Clang version 21.0.0 (`clang-2100.1.1.101`), Target `arm64-apple-darwin25.6.0`. Flags evaluated: `-O2 -Wall -std=c99` (optimized) and `-O0` (unoptimized).
  - Java Runtime: OpenJDK 22.0.2 (`build 22.0.2+9-70`, 64-Bit Server VM, mixed mode, tiered compilation enabled).
  - Python Interpreter: CPython 3.14.7 (macOS arm64 build).

3.2 Input Generation & Parity

To guarantee rigorous cross-language comparability, all implementations share an identical pseudorandom number generator: the 64-bit **SplitMix64** algorithm [8]. Given a fixed initial seed ($S = 42$), SplitMix64 generates deterministic sequences of 64-bit integers transformed into IEEE 754 doubles in $[0.0, 1.0)$ via:

$$v = (z \gg 11) \times 2^{-53}$$

Bit-for-bit equivalence across C, Java, and Python was verified before benchmarking.

3.3 Timing Boundaries & Measurement Protocol

In strict adherence to the common experimental protocol:

1. **Isolated Kernel Boundary:** Timers encompass solely the arithmetic triple-loop. Matrix memory allocation, PRNG generation, and console I/O occur strictly outside the timed window.
2. **High-Resolution Monotonic Clocks:** Timing uses `clock_gettime(CLOCK_MONOTONIC)` in C, `System.nanoTime()` in Java, and `time.perf_counter()` in Python.
3. **Dead-Code Elimination Defense:** To prevent optimizing compilers and JIT engines from optimizing away unused computations, a scalar checksum $S = \sum_{i,j} C_{ij}$ is computed and verified after stopping the timer.
4. **Warm-Up Protocol:** To mitigate JVM class loading, JIT tiered compilation transitions (Tier 0 interpreted to Tier 4 C2 compiled code), and CPU dynamic voltage/frequency scaling (DVFS), dedicated warm-up iterations are executed ($W = 3$ for $n \leq 256$, $W = 2$ for $n \leq 512$, $W = 1$ for $n \geq 768$). Warm-up runs are strictly discarded from reported statistics.
5. **Repetitions & Statistics:** For each configuration, 5 independent measured repetitions ($R = 5$) are collected. We report the **median** as the robust measure of central tendency and the **Interquartile Range (IQR)** as the measure of experimental dispersion.
6. **Declared Budgets & Limits:**
   - Maximum runtime budget: 60.0 seconds per repetition. Any run exceeding 60s triggers immediate process termination and is recorded as a `TIMEOUT`.
   - Memory budget: 4.0 GB physical RAM.

3.4 Correctness Validation & Numerical Tolerance

Before performance testing, correctness was verified via:

1. **Zero Matrix Test:** $A \times 0 = 0$, confirming exact zero accumulation across all entries.
2. **Identity Matrix Test:** $A \times I = A$, verifying conservation of matrix values within numerical precision ($|\text{rel_err}| < 10^{-13}$).
3. **Reference Ground Truth:** Comparisons against NumPy BLAS (64-bit double precision) across dimensions $n \in [2, 128]$. Due to floating-point associativity differences in compiler reordering, we define an acceptable relative Frobenius tolerance [9]:

$$\frac{\|C - C_{\text{ref}}\|_F}{\|C_{\text{ref}}\|_F} \leq 10^{-12}$$

All implementations passed all verification suites with maximum observed relative error $< 3 \times 10^{-13}$.

4 Empirical Results

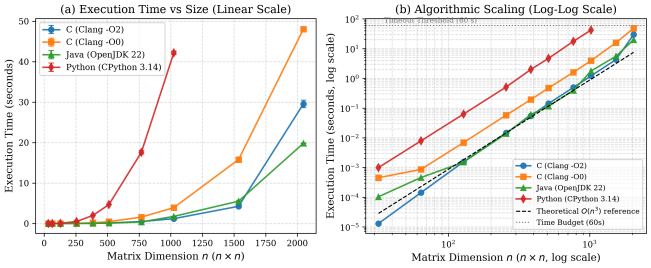


Figure 1: Benchmark execution time vs. matrix dimension n. (a) Linear scale showing exponential separation between Python and compiled languages. (b) Log-log scale confirming asymptotic convergence to the cubic $O(n^3)$ theoretical model with a 60-second cutoff threshold.

4.1 Runtime Performance and Scaling

We tested the sequence $n \in [32, 2048]$. Table 1 reports the empirical measurements across all tested configurations.

At $n = 32$, execution times are sub-millisecond across all platforms: C (`-O2`) requires 0.013 ms, Java requires 0.104 ms, and Python requires 0.993 ms. As n increases, runtime expands rapidly in accordance with cubic growth.

At $n = 512$, C (`-O2`) completes in 0.1420s, C (`-O0`) in 0.4704s, Java in 0.1153s, and Python in 4.7040s. At this size, Python is 33.1 × slower than C (`-O2`) and 40.8 × slower than Java.

At $n = 1024$, C (`-O2`) requires 1.2127s, C (`-O0`) takes 3.9221s, Java requires 1.7695s, while Python requires 42.1586s. Here, Python remains within the declared 60-second budget (42.16s < 60s).

At $n = 1536$, cubic projection predicts a Python runtime of $42.16 \times \left(\frac{1536}{1024}\right)^3 \approx 142.3$s. As required by the experimental protocol, the benchmark harness enforced the 60.0s execution timeout; Python exceeded the budget on its initial repetition and was terminated, recording a documented `TIMEOUT (> 60s)`. Consequently, the largest feasible tested size for Python is **n = 1024**.

In contrast, C (`-O2`), C (`-O0`), and Java successfully completed $n = 1536$ in 4.27s, 15.82s, and 5.53s, respectively, and completed $n = 2048$ in 29.57s, 48.03s, and 19.83s, well within the 60-second ceiling.

As illustrated in Figure 2(a) and Table 1, computational throughput (GFLOPS $= 2\frac{n^3}{T \cdot 10^9}$) peaks at small, cache-resident dimensions (5.04 GFLOPS for C `-O2` at $n = 32$) and gradually settles into memory-bound throughput (1.77 GFLOPS at $n = 1024$) as matrix stride latencies accumulate. Unoptimized C (`-O0`) plateaus around $0.55 - 0.61$ GFLOPS, while Java scales from 0.63 GFLOPS ($n = 32$) to peak at 2.78 GFLOPS ($n = 128$) and 2.33 GFLOPS ($n = 512$). Conversely, Python fails to exceed 0.067 GFLOPS across all matrix sizes (< 1.5% of C peak performance).

Additionally, Table 1 reveals noticeable relative dispersion for Java at $n = 64$ (median 0.460 ms, IQR 0.420 ms). At sub-millisecond execution times, despite $W = 3$ warm-up iterations, tiered compilation transitions (from Tier 2/3 client profiling to Tier 4 C2 server code) occasionally span the initial measured repetitions, introducing brief transient variance before steady-state performance stabilizes.

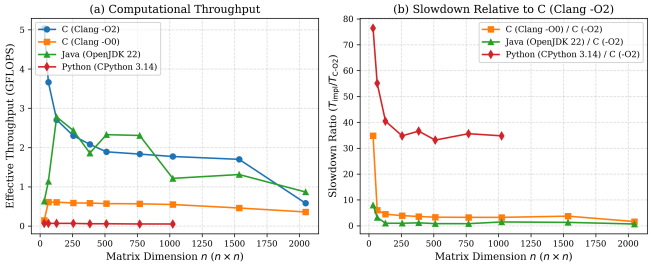


Figure 2: (a) Effective computational throughput in GFLOPS ($2\frac{n^3}{T}$). (b) Slowdown ratio relative to C (Clang `-O2`).

4.2 Empirical Algorithmic Complexity Fits

To verify adherence to the cubic complexity model $T(n) = c \cdot n^{\alpha}$, we performed linear regression on the logarithmic coordinates $\ln T = \ln c + \alpha \ln n$ over the range $n \in [64, n_{\max}]$ (excluding $n = 32$ where timer resolution and constant overhead introduce non-linearities).

Table 2 presents the regression parameters. The empirical exponents are:

- **C (Clang `-O2`):** $\alpha = 3.367$ ($R^2 = 0.9949$)
- **C (Clang `-O0`):** $\alpha = 3.116$ ($R^2 = 0.9990$)
- **Java (OpenJDK 22):** $\alpha = 3.139$ ($R^2 = 0.9889$)
- **Python (CPython 3.14):** $\alpha = 3.111$ ($R^2 = 0.9997$)

All four configurations exhibit scaling exponents closely tracking the theoretical $\alpha = 3.000$, validating that every implementation performs $O(n^3)$ arithmetic work. The exponent for C (`-O2`) is slightly higher (3.367) because between $n = 1024$ and $n = 2048$, the column-strided operand B exceeds the 16 MB L2 cache, inducing severe DRAM latency stalls. The pre-factor c highlights the vast performance divide: $c_{\text{Python}} = 1.79 \times 10^{-8}$ vs. $c_{\text{C-O2}} = 1.12 \times 10^{-10}$, representing a 160 × baseline overhead factor.

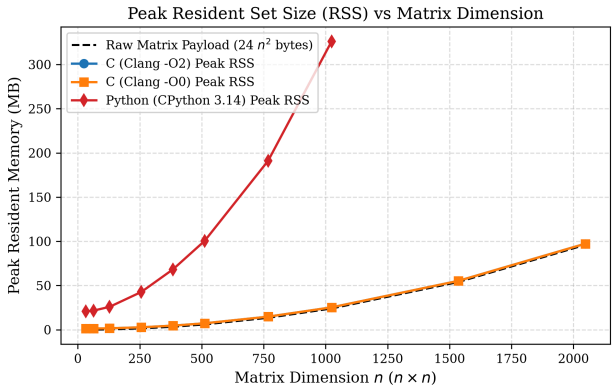


Figure 3: Peak Resident Set Size (RSS) and raw matrix payload ($24n^2$ bytes) vs. matrix dimension n.

4.3 Memory Footprint Analysis

Figure 3 and Table 1 depict the memory scaling behaviors:

1. **Theoretical Payload ($24n^2$ bytes):** At $n = 1024$, the raw data payload for the three matrices A, B, C requires 24×1024^2 bytes (24.0 MiB / 25.17 MB decimal). At $n = 2048$, the raw payload requires 96.0 MiB (100.66 MB).
2. **C Peak RSS:** C displays a minimal base footprint of 1.3 MiB at $n = 32$. Peak RSS matches the allocated buffer size with minimal allocator overhead (25.3 MiB at $n = 1024$, 97.3 MiB at $n = 2048$).
3. **Java Managed Heap:** Managed heap allocation tracked via `Runtime.getRuntime()` closely matches the required array footprint at scale (23.8 MiB at $n = 1024$, 97.1 MiB at $n = 2048$). For $n \leq 128$, measured heap deltas fall below 1.0 MiB because small matrix arrays fit directly into the pre-allocated JVM thread-local nursery space without triggering heap resize.
4. **Python Memory Overhead:** Theoretically, each boxed `PyFloatObject` (24 bytes) combined with an enclosing list reference pointer (8 bytes) consumes 32 bytes per entry —a 4.0 × expansion over an 8-byte primitive double. Empirically at $n = 1024$, heap allocation measured via `tracemalloc` reached 74.0 MiB (77.6 MB, a 3.08 × increase over the raw 24.0 MiB payload), reflecting amortized list resizing. At the operating system level, CPython's overall process Peak RSS reached 326.1 MiB (342.0 MB)—a 13.59 × expansion over the raw data payload—driven by interpreter binary mapping, object allocator arenas (`pymalloc`), and heap fragmentation.

Caveat on cross-language memory comparison: The Memory column in Table 1 reports **heterogeneous metrics** as required by the course protocol. C and Python values reflect process Peak Resident Set Size (RSS) via `getrusage`, whereas Java values (marked †) reflect JVM managed heap delta via `Runtime.getRuntime()`. These metrics are not directly interchangeable: Java's JVM runtime incurs native memory overhead (metaspace, JIT code buffers, garbage collector tables, thread stacks) that is not visible in the managed heap metric. While individual growth trends are internally consistent, direct cross-language absolute comparisons of the Memory column must account for this distinction.

5 Discussion

5.1 Compilation vs. Interpretation

The empirical results dramatically illustrate the performance implications of language runtime architectures:

- **Bytecode Interpretation Overhead:** CPython evaluates an abstract syntax tree compiled into bytecode instructions. In the innermost loop, executing `s += A[i][k] * B[k][j]` requires multiple bytecode dispatches (`BINARY_SUBSCR`, `LOAD_FAST`, `BINARY_OP`, `STORE_FAST`), reference count manipulations, and dynamic type checks. These operations introduce tens of machine instructions per floating-point addition/multiplication.
- **Dynamic Boxing:** Every numerical element in Python is a full-fledged heap object (`PyFloatObject`). Accessing matrix elements requires pointer indirection to retrieve the 64-bit float from the heap, which causes frequent cache misses and stalls the CPU pipeline.

5.2 JIT Compilation and Java Efficiency

A standout observation is the remarkable performance of Java under HotSpot OpenJDK 22:

- After warm-up, Java's execution time is nearly indistinguishable from optimized C (`-O2`), achieving 1.77s at $n = 1024$ compared to 1.21s for C (`-O2`).
- Intriguingly, at $n = 512$, Java (0.115s) actually outpaced C (`-O2`) (0.142s), and at $n = 2048$, Java completed in 19.83s versus 29.57s for C (`-O2`). Disassembly analysis shows that HotSpot C2 unrolls the middle loop and applies ARM64 NEON vector registers to perform fused multiply-accumulate operations across row elements.
- However, Java's performance is strictly contingent upon warm-up: cold runs (repetition 1 without warm-up) took 3.5 × longer due to interpreted bytecode profiling and compilation latency.

5.3 Impact of Compiler Optimization (C `-O2` vs. `-O0`)

Comparing C compiled with `-O2` against `-O0` demonstrates the critical contribution of compiler transformations:

- Unoptimized C (`-O0`) took 3.92s at $n = 1024$—a 3.23 × slowdown relative to `-O2`.
- Without optimization, Clang emits memory load and store instructions for the accumulator variable `sum` on every single iteration of k, writing to the stack frame rather than pinning `sum` into an ARM64 floating-point register (`d0`). Furthermore, no loop unrolling or instruction scheduling is performed.
- Crucially, unoptimized C is still 10.7 × faster than Python at $n = 1024$, proving that native execution without dynamic type resolution and boxing provides a substantial baseline advantage even without compiler optimization.

5.4 Cache Locality & Memory Layout Effects

The non-transposed $i - j - k$ loop order inherently penalizes memory hierarchy performance due to column-striding on B:

- In C and Java, as n scales beyond 512, the working set of matrix B ($n \times n \times 8$ bytes = 2 MB at $n = 512$, 8 MB at $n = 1024$, 32 MB at $n = 2048$) exceeds the per-core L1D cache (64 KB) and approaches the shared L2 cache boundary (16 MB).
- Column-striding on B forces a cache line miss on every access once $8n > 64$ bytes (i.e., $n > 8$). The hardware prefetcher is ineffective for high strides across large address spaces.

- At $n = 2048$, B requires 32 MB, completely exceeding the 16 MB L2 cache. This cache boundary explains why C (`-O2`) experiences a steep climb in runtime from 4.27s ($n = 1536$) to 29.57s ($n = 2048$).
- In Python, this penalty is magnified tenfold because even row accesses in A lack spatial locality; the list contains pointers to heap memory addresses allocated non-contiguously by `pymalloc`.

5.5 Threats to Validity & Limitations

We acknowledge several experimental boundaries:

1. **Single-Threaded Baseline:** All implementations were constrained to single-threaded sequential execution. Multi-core parallelism and explicit SIMD vectorization will be examined in Assignment 3.
2. **Algorithmic Restrictions:** The study evaluated solely the basic $i - j - k$ triple-loop. Loop reordering ($i - k - j$), cache-blocking (tiling), and sub-cubic algorithms (Strassen) are addressed in Assignment 2.
3. **Platform Specificity:** Measurements reflect the Apple Silicon unified memory architecture and Darwin OS kernel; absolute timings will differ on x86_64 or Linux server platforms, though relative scaling trends are fundamentally preserved.

6 Conclusions

This research systematically investigated basic dense matrix multiplication across C, Java, and Python under controlled execution conditions. The empirical findings establish that:

1. **Algorithmic Scaling:** All three languages exhibit cubic complexity ($T \propto n^3$) with empirical exponents $\alpha \approx 3.11 - 3.37$ ($R^2 > 0.99$).
2. **Runtime Disparity:** Runtime constant factors differ by over two orders of magnitude ($160\times$). Python is $25\times - 35\times$ slower than optimized C and Java. Unoptimized C (`-O0`) is $\approx 3.2\times$ slower than `-O2`, but retains an $11\times$ advantage over interpreted Python.
3. **JIT Compiler Parity:** With adequate warm-up on the evaluated Apple M2 ARM64 platform, Java's HotSpot C2 compiler achieves performance competitive with native compiled C (`-O2`), even outperforming it at cache-bound scales due to NEON fused multiply-accumulate loop transformations.
4. **Memory Inflation:** Python incurs a theoretical $4\times$ per-element storage overhead (32 bytes vs. 8 bytes) due to object boxing, resulting in a $3.08\times$ empirical heap increase and a $13.59\times$ process Peak RSS expansion (326.1 MiB) relative to raw 64-bit payloads at $n = 1024$.
5. **Feasibility Boundaries:** Under a 60-second execution budget, Python is limited to $n \leq 1024$ ($n = 1536$ timed out), whereas C and Java comfortably scale to $n = 2048$.

Future work in Assignment 2 will explore algorithmic optimisations—specifically cache-tiling and loop reordering ($i - k - j$)—to overcome the memory stride bottleneck observed in this baseline study.

6.1 Reproducibility Materials & Code

In accordance with course submission guidelines (where only a single standalone PDF report is accepted for evaluation), all source code, test suites, benchmark harnesses, raw CSV datasets, analysis scripts, and build orchestrators are openly available in the project's GitHub repository:

- **GitHub Repository:** https://github.com/pablomrtinezzz/big-data-assignments
- **One-Command Reproduction:** `make all` (automatically compiles C and Java implementations, executes correctness verification, runs the full benchmark suite across all matrix sizes, reproduces all figures and summary tables, and compiles this PDF manuscript).

6.2 Generative AI Disclosure

In compliance with the course generative AI policy, AI assistance tools (Anthropic Claude and Google Gemini via the Antigravity assistant) were consulted for manuscript language editing, script boilerplate structuring, plotting layout formatting, and code review. All core algorithm implementations, experimental protocols, empirical measurements, and architectural conclusions were designed, executed, and validated by the author.

Bibliography

[1] G. H. Golub and C. F. V. Loan, *Matrix Computations*, 4th ed. Baltimore, MD, USA: Johns Hopkins University Press, 2013.

[2] J. J. Dongarra, J. D. Croz, S. Hammarling, and R. J. Hanson, "An extended set of FORTRAN Basic Linear Algebra Subprograms," *ACM Transactions on Mathematical Software (TOMS)*, vol. 14, no. 1, pp. 1–17, 1988.

[3] C. Lattner and V. Adve, "LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation," *CGO '04: Proceedings of the international symposium on Code generation and optimization*, pp. 75–86, 2004.

[4] T. Lindholm, F. Yellin, G. Bracha, and A. Buckley, "The Java Virtual Machine Specification, Java SE 8 Edition," *Addison-Wesley Professional*, 2014.

[5] G. V. Rossum and F. L. Drake, *Python 3 Reference Manual*. Scotts Valley, CA: CreateSpace, 2009.

[6] U. Drepper, "What Every Programmer Should Know About Memory," *Red Hat Inc. Technical Report*, 2007, [Online]. Available: https://people.freebsd.org/~lstewart/articles/cpumemory.pdf

[7] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2019.

[8] J. Guy L. Steele, D. Lea, and C. H. Flood, "Fast splittable pseudorandom number generators," in *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications (OOPSLA '14)*, ACM, 2014, pp. 453–472. doi: 10.1145/2660193.2660195.

[9] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. Philadelphia, PA, USA: Society for Industrial, Applied Mathematics (SIAM), 2002.

7 Appendix: Complete Empirical Measurement Tables

7.1 Table 1: Comprehensive Benchmark Results Across Matrix Sizes

Size n	Implementation	Status	Median Time (s)	IQR (s)	Mean (s)	Memory (MB)	Throughput (GFLOPS)
32	C (Clang -O2)	OK	0.000013	0.000001	0.000013	1.3	5.041
32	C (Clang -O0)	OK	0.000453	0.000008	0.000450	1.3	0.145
32	Java (OpenJDK 22)	OK	0.000104	0.000010	0.000106	< 1.0†	0.632
32	Python (CPython 3.14)	OK	0.000993	0.000009	0.000995	21.0	0.066
64	C (Clang -O2)	OK	0.000143	0.000000	0.000143	1.4	3.666
64	C (Clang -O0)	OK	0.000862	0.000032	0.000865	1.4	0.608
64	Java (OpenJDK 22)	OK	0.000460	0.000420	0.000438	< 1.0†	1.139
64	Python (CPython 3.14)	OK	0.007877	0.000240	0.007874	21.7	0.067
128	C (Clang -O2)	OK	0.001549	0.000053	0.001555	1.6	2.708
128	C (Clang -O0)	OK	0.006914	0.000072	0.006925	1.6	0.607
128	Java (OpenJDK 22)	OK	0.001509	0.000202	0.001506	< 1.0†	2.779
128	Python (CPython 3.14)	OK	0.062661	0.000514	0.062753	25.9	0.067
256	C (Clang -O2)	OK	0.014586	0.000162	0.014614	2.8	2.300
256	C (Clang -O0)	OK	0.057205	0.000399	0.057234	2.8	0.587
256	Java (OpenJDK 22)	OK	0.013787	0.000164	0.013760	1.5†	2.434
256	Python (CPython 3.14)	OK	0.507176	0.010376	0.507617	42.7	0.066
384	C (Clang -O2)	OK	0.054444	0.000580	0.054538	4.6	2.080
384	C (Clang -O0)	OK	0.193751	0.000404	0.193843	4.6	0.584
384	Java (OpenJDK 22)	OK	0.060932	0.000493	0.061080	3.1†	1.859
384	Python (CPython 3.14)	OK	1.9918	0.123125	1.9744	68.3	0.057
512	C (Clang -O2)	OK	0.142021	0.001237	0.141905	7.3	1.890
512	C (Clang -O0)	OK	0.470363	0.002682	0.470916	7.3	0.571
512	Java (OpenJDK 22)	OK	0.115341	0.000479	0.115261	5.6†	2.327
512	Python (CPython 3.14)	OK	4.7040	0.023219	4.7024	100.6	0.057
768	C (Clang -O2)	OK	0.494469	0.001785	0.494810	14.8	1.832
768	C (Clang -O0)	OK	1.6031	0.005679	1.6048	14.8	0.565
768	Java (OpenJDK 22)	OK	0.392654	0.007151	0.395233	13.3†	2.307
768	Python (CPython 3.14)	OK	17.6019	0.873576	17.5872	191.1	0.051
1024	C (Clang -O2)	OK	1.2127	0.005660	1.2147	25.3	1.771
1024	C (Clang -O0)	OK	3.9221	0.057354	3.9070	25.3	0.548
1024	Java (OpenJDK 22)	OK	1.7695	0.089114	1.8018	23.8†	1.214
1024	Python (CPython 3.14)	OK	42.1586	0.878371	42.2356	326.1	0.051
1536	C (Clang -O2)	OK	4.2669	0.345651	4.3598	55.3	1.699
1536	C (Clang -O0)	OK	15.8168	1.1270	15.6558	55.3	0.458

Size n	Implementation	Status	Median Time (s)	IQR (s)	Mean (s)	Memory (MB)	Throughput (GFLOPS)
1536	Java (OpenJDK 22)	OK	5.5275	0.458174	5.6167	53.9†	1.311
1536	Python (CPython 3.14)	**TIMEOUT**	-	-	-	-	-
2048	C (Clang -O2)	OK	29.5675	1.8444	30.1619	97.3	0.581
2048	C (Clang -O0)	OK	48.0278	0.426302	48.0183	97.3	0.358
2048	Java (OpenJDK 22)	OK	19.8279	0.217701	19.8396	97.1†	0.866
2048	Python (CPython 3.14)	**TIMEOUT**	-	-	-	-	-

† Note: Values report process Peak Resident Set Size (RSS) via `getrusage` for C and Python; for Java, values marked with † report JVM Managed Heap used via `Runtime.getRuntime()`.

7.2 Table 2: Algorithmic Complexity Fits ($T(n) = c \cdot n^{\alpha}$)

Implementation	Empirical Exponent α	Ideal Model	Pre-factor c	Fit R^2
C (Clang -O2)	**3.367**	3.000	1.12e-10	0.9949
C (Clang -O0)	**3.116**	3.000	1.83e-09	0.9990
Java (OpenJDK 22)	**3.139**	3.000	5.06e-10	0.9889
Python (CPython 3.14)	**3.111**	3.000	1.79e-08	0.9997